

Hotel Management System

Software Requirements Specification

Prepared in accordance with IEEE Std 830-1998 guidelines

Prepared for: [Client / Hotel Group Name]

Prepared by: Disura

Date: July 2026

Version: 1.0

Status: Draft for client review

Revision History

Version	Date	Author	Description
0.1	18 Jun 2026	Disura	Initial draft of functional requirements
1.0	01 Jul 2026	Disura	Complete SRS issued for review

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Hotel Management System (HMS). It is intended for the client/product owner, the development team, and any future maintainers of the system, and follows the general structure recommended by IEEE Std 830-1998 for software requirements documents.

1.2 Scope

The HMS is a web-based platform that enables hotel staff to manage room inventory, reservations, guest profiles, billing, housekeeping, and reporting across one or more properties. It also exposes a guest-facing booking widget for direct reservations. The system is designed to be cost-effective to host, straightforward to maintain, and secure enough for production use, including handling of guest payment data via a third-party payment gateway.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
HMS	Hotel Management System – the system described in this document
PMS	Property Management System – industry term for hotel operations software
Folio	A guest's running account of charges and payments during a stay
RBAC	Role-Based Access Control
OTA	Online Travel Agency (e.g., Booking.com, Agoda, Expedia)
ADR	Average Daily Rate – average revenue earned per occupied room per day
RevPAR	Revenue Per Available Room
Night audit	End-of-day process that closes out the day's transactions and rolls the system date forward

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Project Proposal – Hotel Management System, v1.0.
- Software Design Document – Hotel Management System, v1.0.
- Database Design Document – Hotel Management System, v1.0.
- PCI DSS v4.0 – Payment Card Industry Data Security Standard (referenced for payment-handling requirements).

1.5 Overview

Section 2 gives an overall description of the product, its users and constraints. Section 3 lists specific functional and non-functional requirements, organised by module. Section 4 describes external interface requirements. Section 5 lists use cases in detail.

2. Overall Description

2.1 Product Perspective

The HMS is a new, self-contained product. It is not a replacement module within an existing legacy system, though it is designed to allow future integration with OTAs and accounting software via its API layer. It follows a client-server architecture: browser and mobile clients communicate with a backend API over HTTPS, which in turn communicates with a relational database and third-party services (payment gateway, email/SMS provider).

2.2 Product Functions (Summary)

- Room and rate management (room types, individual rooms, seasonal rates).
- Reservation management (search availability, create/modify/cancel bookings).
- Guest profile management (contact details, ID/passport, stay history).
- Check-in / check-out workflow.
- Folio and billing management with integrated payment processing.
- Housekeeping status tracking.
- Role-based staff accounts and permissions.
- Operational and financial reporting.
- Multi-property support for hotel groups.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Owner / General Manager	Full access across all properties; views financial reports, manages staff and rates	Low to moderate
Branch Manager	Full operational access for their assigned property/properties	Low to moderate
Front Desk / Cashier	Creates and manages reservations, checks guests in/out, records payments	Low
Housekeeping Staff	Updates room cleaning/maintenance status only	Low
Accountant	Views billing, payments and financial reports; no reservation editing	Low to moderate
Guest (public)	Uses the public booking widget to search and reserve rooms	None assumed

2.4 Operating Environment

- Server-side: Linux-based cloud environment (containerised), Node.js runtime, PostgreSQL database.
- Client-side (staff): modern desktop browser (Chrome, Edge, Firefox, Safari, latest two major versions).
- Client-side (guest booking widget): modern desktop and mobile browsers.

- Optional: Electron-based front-desk terminal application for offline-tolerant kiosk use.

2.5 Design and Implementation Constraints

- The system must isolate data by property to support multi-branch clients from day one.
- All card payment data must be handled exclusively via a PCI-DSS-compliant third-party gateway; the system must never store raw card numbers.
- The system must remain operable on cost-effective infrastructure suitable for a small-to-mid-sized hotel budget.
- Time-sensitive operations (night audit, checkout timing) must correctly account for the property's local time zone.

2.6 Assumptions and Dependencies

- The client will provide a merchant account with the selected payment gateway (Stripe or a local aggregator such as PayHere).
- The client will provide branding assets (logo, color scheme) for the guest-facing booking widget.
- Reliable internet connectivity is available at each property for staff to access the system.
- OTA channel synchronisation is treated as a future phase and is not a dependency of the Phase 1 release.

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Authentication and Access Control

ID	Requirement	Priority
FR-1.1	The system shall allow staff to log in using an email/username and password.	High
FR-1.2	The system shall enforce role-based access control, restricting each API endpoint and UI view according to the logged-in user's role.	High
FR-1.3	The system shall support optional two-factor authentication (TOTP) for Owner and Manager roles.	Medium
FR-1.4	The system shall lock an account after 5 consecutive failed login attempts for 15 minutes.	Medium
FR-1.5	The system shall allow an Owner/Manager to create, edit, deactivate, and assign roles to staff accounts.	High

3.1.2 Room and Rate Management

ID	Requirement	Priority
FR-2.1	The system shall allow authorised staff to define room types (e.g., Standard, Deluxe, Suite) with a base rate and maximum occupancy.	High
FR-2.2	The system shall allow authorised staff to add, edit, and deactivate	High

ID	Requirement	Priority
	individual rooms under a room type.	
FR-2.3	The system shall allow authorised staff to define rate plans (e.g., seasonal rate, non-refundable rate) per room type.	High
FR-2.4	The system shall prevent two confirmed reservations from overlapping on the same physical room, enforced at the database level.	High

3.1.3 Reservation Management

ID	Requirement	Priority
FR-3.1	The system shall allow searching room availability by property, date range, and guest count.	High
FR-3.2	The system shall allow staff to create a reservation on behalf of a guest, including walk-ins.	High
FR-3.3	The system shall allow guests to create a reservation through the public booking widget.	High
FR-3.4	The system shall allow staff to modify or cancel an existing reservation, subject to the applicable cancellation policy.	High
FR-3.5	The system shall send a booking confirmation email to the guest upon successful reservation.	Medium
FR-3.6	The system shall display a real-time occupancy/availability calendar to front-desk staff.	High

3.1.4 Check-in / Check-out

ID	Requirement	Priority
FR-4.1	The system shall allow front-desk staff to check in a guest against an existing reservation, capturing ID/passport details.	High
FR-4.2	The system shall automatically update room status to “Occupied” upon check-in and “Dirty” upon checkout.	High
FR-4.3	The system shall allow staff to check out a guest only once the folio balance is zero or a payment method is confirmed.	High

3.1.5 Billing and Payments

ID	Requirement	Priority
FR-5.1	The system shall maintain a folio per reservation, itemising all charges (room, services, taxes).	High
FR-5.2	The system shall allow staff to post ad-hoc charges (e.g., minibar, laundry) to a guest's folio.	High
FR-5.3	The system shall integrate with a third-party payment gateway to process card payments.	High

ID	Requirement	Priority
FR-5.4	The system shall record all payments and refunds against the relevant folio with a timestamp and staff identifier.	High
FR-5.5	The system shall generate a printable/emailable invoice upon checkout.	Medium

3.1.6 Housekeeping

ID	Requirement	Priority
FR-6.1	The system shall allow housekeeping staff to update a room's status (clean, dirty, inspected, out-of-order).	High
FR-6.2	The system shall prevent a room marked “out-of-order” from appearing in availability search results.	High

3.1.7 Reporting

ID	Requirement	Priority
FR-7.1	The system shall generate an occupancy report for a selected date range and property.	High
FR-7.2	The system shall generate a daily revenue and night-audit summary report.	High
FR-7.3	The system shall calculate and display ADR and RevPAR for a selected period.	Medium
FR-7.4	The system shall allow exporting reports to CSV/PDF.	Medium

3.1.8 Multi-Property Support

ID	Requirement	Priority
FR-8.1	The system shall allow an Owner to manage multiple properties under a single account.	High
FR-8.2	The system shall scope all reservation, room, and billing data strictly to its owning property.	High
FR-8.3	The system shall allow an Owner/Manager to view consolidated reports across all properties.	Medium

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	95% of API requests shall complete within 500ms under normal load (excluding third-party payment calls).
NFR-2	Availability	The production system shall maintain 99.5% uptime outside scheduled maintenance windows.
NFR-3	Security	All traffic shall be encrypted via HTTPS/TLS 1.2 or higher.
NFR-4	Security	Passwords shall be hashed using bcrypt or argon2; plaintext

ID	Category	Requirement
		passwords shall never be stored or logged.
NFR-5	Security	Raw payment card data shall never be stored on HMS servers; tokenisation via the payment gateway shall be used exclusively.
NFR-6	Auditability	All rate overrides, refunds, cancellations, and role changes shall be recorded in an audit log with user, timestamp, and action.
NFR-7	Usability	A new front-desk staff member shall be able to complete a check-in without developer assistance after a 1-hour training session.
NFR-8	Scalability	The system shall support horizontal scaling of the application layer without code changes.
NFR-9	Maintainability	The codebase shall use a single primary language (TypeScript) across frontend and backend to reduce long-term maintenance overhead.
NFR-10	Localisation	The system shall correctly handle date/time calculations in the property's local time zone (e.g., Sri Lanka, UTC+5:30), including across daylight-saving boundaries where applicable.
NFR-11	Backup & Recovery	The production database shall be backed up at least daily, with point-in-time recovery available for at least 7 days.
NFR-12	Compatibility	The staff dashboard shall function correctly on the latest two major versions of Chrome, Edge, Firefox, and Safari.

4. External Interface Requirements

4.1 User Interfaces

The system provides a staff web dashboard (desktop-first, responsive to tablet) and a guest-facing booking widget (desktop and mobile responsive). Detailed wireframes and interaction flows are provided in the UI/UX Design Document.

4.2 Hardware Interfaces

No specialised hardware is required. The system is designed to run on standard desktop/laptop computers and tablets with a modern web browser. Optional receipt printers may be integrated via standard OS print drivers.

4.3 Software Interfaces

Interface	Purpose
Payment gateway API (Stripe / PayHere)	Processing card payments and refunds
Email/SMS provider API	Sending booking confirmations and notifications
PostgreSQL	Primary data store
Redis	Caching, session storage, background job queue

4.4 Communications Interfaces

All client-server communication occurs over HTTPS using a RESTful JSON API. Real-time availability updates on the booking calendar are delivered via a WebSocket channel.

5. Use Case Model

The diagram below summarises the primary actors and use cases of the system.

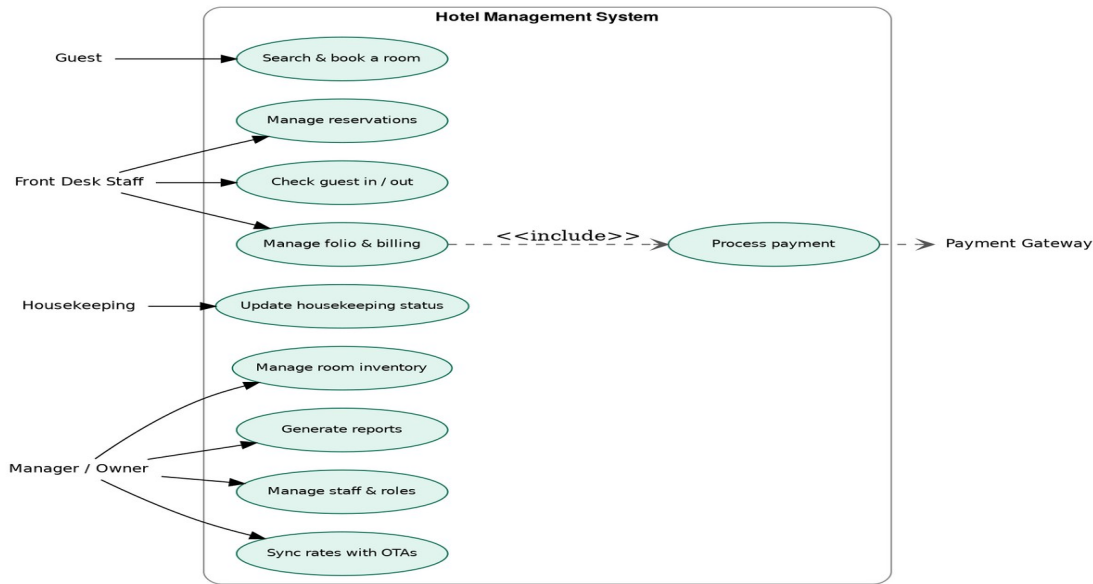


Figure 1: Use case diagram – Hotel Management System

5.1 Use Case: Book a Room (Guest, via booking widget)

Field	Description
Actor	Guest
Precondition	Guest has selected valid check-in/check-out dates
Main flow	1. Guest searches availability. 2. System returns available room types and rates. 3. Guest selects a room and enters details. 4. Guest completes payment via the payment gateway. 5. System confirms the reservation and sends a confirmation email.
Alternate flow	If no rooms are available for the selected dates, the system displays alternative dates or room types.
Postcondition	A confirmed reservation exists; the room is no longer available for the booked dates.

5.2 Use Case: Check-in a Guest (Front Desk)

Field	Description
Actor	Front Desk Staff

Field	Description
Precondition	A confirmed reservation exists for the current date
Main flow	1. Staff searches for the reservation. 2. Staff verifies guest identity and captures ID details. 3. Staff assigns a specific room if not already assigned. 4. System updates room status to Occupied and opens the guest folio.
Postcondition	Guest is checked in; room status is Occupied.

5.3 Use Case: Process Checkout Payment (Front Desk)

Field	Description
Actor	Front Desk Staff, Payment Gateway
Precondition	Guest folio has an outstanding balance
Main flow	1. Staff opens the guest folio. 2. Staff selects payment method. 3. System sends a payment request to the gateway. 4. Gateway confirms payment via webhook. 5. System updates the folio balance and marks the reservation as checked out.
Exception flow	If the payment is declined, the system displays an error and the folio balance remains unpaid.
Postcondition	Folio balance is zero; room status is Dirty (pending housekeeping).

Appendix A: Requirements Traceability

Each functional requirement above is designed to map directly to a corresponding module in the Software Design Document and a set of test cases to be defined during the testing phase, ensuring every requirement is both implemented and verified before go-live.